future authorization decisions. These actions don’t require writing code or modifying a policy file; they’re made through contextual UI components that reflect the resource currently being viewed. This tight coupling between user intent, interface, and authorization data makes ReBAC uniquely flexible and delegatable. Anyone can be a policy editor—not by crafting rules, but by shaping relationships. Additionally, a graph management tool will likely be available, enabling administrators to access the graph for debugging, auditing, and other global activities.

The Policy Information Point (PIP), by contrast, supplies the decision engine with the data it needs to evaluate access. In a ReBAC system, that includes the relationship graph but may also involve other sources—such as the identity provider—to answer questions like “who is this principal?” or “what customer does this employee work with?”

6.1.3 Strengths and limitations of ReBAC

ReBAC offers a powerful model for managing access in complex, collaborative systems. *Its main idea—that relationships influence access choices—makes it especially useful for systems where authorization depends on social, organizational, or ownership structures that often change.* Rather than embed access rules in fixed roles or code policies, ReBAC relies on a relationship graph as the source of truth. If a principal is connected to a resource via an authorized path, access is granted.

By treating data as policy, ReBAC enables teams to delegate policy authoring to the people closest to the work. Permissions are granted and revoked through intuitive UI actions—such as inviting a teammate, assigning ownership, or sharing a workspace—without involving centralized administrators. This makes ReBAC highly scalable and adaptable to change, especially in areas where resource boundaries and team structures often shift. On the other hand, delegating policy authority to individuals requires trusting them to make informed decisions. This informs the use cases where ReBAC makes sense.

But ReBAC has its tradeoffs. In large or densely connected graphs, it can be difficult to see the impact of a single change on access. Debugging who has access—and why—may require tracing complex relationship paths, often